

\$ Aide-mémoire Docker

Les commandes essentielles, groupées par usage. Chaque partie d'une commande garde toujours la même couleur pour qu'on la repère d'un coup d'œil.

■ commande racine ■ sous-commande ■ option / flag ■ valeur à remplacer

Conteneurs et images — les bases

```
docker ps
```

Liste les conteneurs **actifs**, avec pour chacun : ID, image, commande de démarrage, date de création, statut (ex. Up 3 minutes), ports publiés (ex. 0.0.0.0:8082->8082/tcp = port hôte → port conteneur) et nom.

```
docker ps -a
```

Liste **tous** les conteneurs, y compris ceux arrêtés (utile pour les redémarrer).

```
docker start <container>
```

Démarre un conteneur arrêté.

```
docker stop <container>
```

Arrête un conteneur en cours d'exécution.

```
docker rm <container>
```

Supprime un conteneur.

```
docker exec -it <container> bash
```

Ouvre un terminal interactif à l'intérieur d'un conteneur déjà en cours d'exécution (-i = interactif, -t = alloue un pseudo-terminal). Utile pour inspecter le conteneur ou y lancer une commande ponctuelle.

Conseil : les images minimales (ex. basées sur Alpine) n'ont souvent pas bash ; utilise alors sh : `docker exec -it <container> sh`. L'invite affichée (ex. /app #) indique le dossier de travail du conteneur.

```
docker inspect <container> | less
```

Affiche tous les détails d'un conteneur au format JSON (configuration, réseau, montages, variables d'environnement...). Fonctionne aussi sur une image, un volume ou un réseau. | less permet de parcourir une sortie longue page par page.

```
docker exec -it mysql-docker-container bash    # 1. entre dans le conteneur
mysql -u root -p                             # 2. lance le client MySQL (demande le mot de
passe)
```

```
show databases;  
exit  
exit
```

```
# 3. liste les bases présentes  
# 4. quitte le client mysql  
# 5. quitte le shell bash du conteneur
```

Exemple complet : se connecter à une base MySQL qui tourne dans un conteneur, l'interroger, puis ressortir proprement.

```
docker run <image>
```

Crée un nouveau conteneur à partir de l'image et le démarre (télécharge l'image si elle n'est pas déjà en local).

```
docker images
```

Liste les images présentes en local.

```
docker rmi <image_ou_id>
```

Supprime une image.

```
docker pull <image>
```

Télécharge une image depuis un registre (ex. Docker Hub), sans créer de conteneur.

Options avancées de run

```
docker run -p 8082:8082 <image>
```

Publie un port : hôte:conteneur. Le port de gauche est celui exposé sur la machine hôte.

```
docker run --name <nom> <image>
```

Nomme le conteneur au lieu d'un nom aléatoire généré automatiquement. Ce nom sert ensuite directement pour stop, start et rm.

```
docker run -d <image>
```

Lance le conteneur en mode **détaché** (arrière-plan) : le terminal reste libre.

```
docker run -e VAR=valeur
```

Définit une variable d'environnement dans le conteneur. Répétable pour en passer plusieurs (ex. MYSQL_ROOT_PASSWORD, MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD pour une image MySQL).

```
docker run --network host
```

Fait partager au conteneur la pile réseau de la machine hôte : pas d'IP propre, les ports du conteneur sont directement ceux de l'hôte (rend -p inutile).

```
<image> --arg=valeur
```

Ce qui suit le nom de l'image est passé en argument à l'application dans le conteneur, pas à Docker. Ex. `--spring.profiles.active=prod` active un profil Spring Boot.

```
docker run -p 8082:8082 --name mon-conteneur -d mon-image:1.0.0
```

Exemple combinant les options ci-dessus.

```
docker run -e MYSQL_ROOT_PASSWORD=root_password -e MYSQL_DATABASE=my_database -e  
MYSQL_USER=app_user -e MYSQL_PASSWORD=app_password --network host mysql:latest
```

Exemple : lance MySQL avec ses identifiants et sa base initiale, en réseau hôte.

```
docker rm mon-service  
docker run --name mon-service -p 8082:8082 mon-service:2.0.0 --spring.profiles.active=prod
```

Exemple : remplace un conteneur existant par une nouvelle version de l'image, avec un profil Spring actif.

```
docker run -e SPRING_PROFILES_ACTIVE=<profil>
```

Variable d'environnement reconnue par Spring Boot : active un profil (ex. `test`, `prod`). Équivalent à passer `--spring.profiles.active=<profil>` en argument après l'image — deux façons de faire la même chose.

```
docker build -f Dockerfile-name -t mon-service:2.0.1 .  
docker run --name mon-service -e SPRING_PROFILES_ACTIVE=test -p 8082:8082 mon-service:2.0.1
```

Exemple : builder depuis un Dockerfile spécifique, puis lancer avec un profil de test.

```
docker run -e server.port=<port>
```

Redéfinit une propriété Spring Boot (ici `server.port`) via une variable d'environnement nommée exactement comme la propriété. Spring la reconnaît directement, sans passer par la forme habituelle `SERVER_PORT`. Le port choisi doit correspondre au port conteneur donné à `-p`.

```
docker run --name mon-service -e server.port=8081 -p 8081:8081 -e  
SPRING_PROFILES_ACTIVE=test mon-service:2.0.1
```

Exemple : port et profil définis via des variables d'environnement (le port doit être cohérent avec `-p`).

```
docker run --env-file <fichier>
```

Charge toutes les variables d'environnement depuis un fichier (une par ligne, format `VAR=valeur`), au lieu de les lister une par une avec plusieurs `-e`.

```
docker run --name mon-service -p 8081:8081 --env-file environment.env mon-service:2.0.1
```

Exemple : charger les variables depuis un fichier plutôt que de les passer une à une.

```
docker run --rm
```

Supprime automatiquement le conteneur dès qu'il s'arrête. Évite d'accumuler des conteneurs arrêtés, pratique pour un test ponctuel.

```
docker run --rm alpine
```

Exemple : lance un conteneur jetable, supprimé automatiquement à la sortie.

Build et gestion des images

```
docker build -t nom:version .
```

Construit une image depuis le Dockerfile du dossier courant, et la nomme (tag `nom:version`).

```
docker build -f <Dockerfile> -t nom:version .
```

Précise le fichier à utiliser comme Dockerfile (utile s'il y en a plusieurs, ex. un fichier `Dockerfile-name`). Sans `-f`, Docker cherche un fichier nommé `Dockerfile`.

```
docker image prune -a
```

Supprime toutes les images non utilisées par un conteneur en une seule fois.

Précision : sans `-a`, seules les images « dangling » (sans tag) sont supprimées. Avec `-a`, toute image non référencée par un conteneur part aussi — ce n'est pas littéralement « toutes les images » du système.

Filtrage et nettoyage

```
docker ps -a --filter "clé=valeur"
```

Liste les conteneurs en appliquant un filtre. Répéter `--filter` combine les filtres (ET logique).

```
docker ps -a --filter "ancestor=mon-service:2.0.0"
```

Filtre les conteneurs créés à partir de l'image `mon-service:2.0.0`.

```
docker ps -a --filter "status=exited"
```

Filtre les conteneurs arrêtés.

```
... -q | xargs docker rm
```

Ajouté à la suite d'un filtrage : supprime en une fois tous les conteneurs correspondants (`-q` ne renvoie que les IDs, `xargs` les passe un à un à `docker rm`).

Logs

```
docker logs <container>
```

Affiche les logs du conteneur.

```
docker logs -f <container>
```

Affiche les logs en continu (« follow »), utile pour suivre l'activité en direct.

Volumes

Un volume nommé permet de conserver les données d'un conteneur même s'il est supprimé (contrairement au système de fichiers du conteneur, qui disparaît avec lui). Docker le stocke sur l'hôte, dans `/var/lib/docker/volumes/<nom>/_data`.

```
docker volume create <nom>
```

Crée un volume nommé, géré par Docker.

```
docker run -v <volume>:<chemin_conteneur> <image>
```

Monte le volume à l'emplacement indiqué dans le conteneur.

```
docker volume ls
```

Liste tous les volumes existants sur la machine. Un volume nommé apparaît sous son nom (ex. `my-app-volume`) ; un volume anonyme apparaît sous un identifiant de 64 caractères généré par Docker.

```
docker volume inspect <volume>
```

Affiche le détail d'un volume au format JSON : date de création, driver, point de montage sur l'hôte (Mountpoint), etc. Pour un volume anonyme, le label `com.docker.volume.anonymous` confirme qu'il n'a pas de nom choisi.

```
docker inspect my-app-volume
```

Exemple : `docker inspect` fonctionne aussi directement sur un volume (même résultat que `docker volume inspect`). Le champ Mountpoint indique où sont stockées les données sur l'hôte, ici `/var/lib/docker/volumes/my-app-volume/_data`.

Conseil : avec Docker Desktop (Windows, macOS), ce chemin se trouve dans la machine virtuelle de Docker, pas directement sur ton disque. Pour voir le contenu, passe par un conteneur jetable (voir l'astuce en fin de section).

```
docker volume create my-app-volume
```

```
docker run -v my-app-volume:/my-app/data mon-service:2.0.1
```

Exemple : crée le volume `my-app-volume` puis le monte dans le conteneur.

```
docker run -p 8082:8082 --name mon-service -e SPRING_PROFILES_ACTIVE=test -v my-app-volume:/app/logs mon-service:2.0.1
```

Exemple : attacher un volume existant à un nouveau conteneur. Le volume `my-app-volume` a déjà été créé (et

éventuellement utilisé par un conteneur précédent) : le nouveau conteneur le monte sur `/app/logs` et retrouve les données qu'il contient déjà, puis y ajoute ses propres logs. Ici avec en plus un port publié, un conteneur nommé et le profil Spring `test`.

Conseil : si le nom de volume donné à `-v` n'existe pas encore, Docker le crée automatiquement (vide) au lieu de signaler une erreur. Vérifie l'orthographe avec `docker volume ls` pour être sûr de réutiliser le bon volume.

```
docker volume rm <volume>
```

Supprime un volume et toutes les données qu'il contient. Accepte un nom ou l'identifiant d'un volume anonyme, et plusieurs volumes à la suite.

Conseil : un volume utilisé par un conteneur, même arrêté, ne peut pas être supprimé. Supprime d'abord le conteneur avec `docker rm`.

```
docker volume rm my-app-volume
```

Exemple : supprime le volume nommé `my-app-volume`. Vérifie ensuite avec `docker volume ls` qu'il n'apparaît plus dans la liste.

Volume nommé en lecture seule

Un volume nommé peut être monté en **lecture seule** : le conteneur peut lire les fichiers, mais ne peut ni les modifier ni les supprimer. Utile pour garantir que des données (typiquement une configuration) ne changent pas pendant l'exécution.

```
docker run -v <volume>:<chemin_conteneur>:ro <image>
```

Le suffixe `:ro` (*read-only*) à la fin de `-v` monte le volume en lecture seule. Sans suffixe, le volume est en lecture-écriture.

```
docker run -d -p 8080:8080 \
-v tomcat-config:/usr/local/tomcat/conf:ro \
-v tomcat-logs:/usr/local/tomcat/logs \
tomcat:latest
```

Exemple avec Tomcat : la configuration (`server.xml`, `tomcat-users.xml`, certificats TLS...) est montée en lecture seule pour éviter toute modification accidentelle ou non autorisée, tandis que le volume des logs reste en écriture pour que Tomcat puisse créer et mettre à jour ses fichiers de log. Le `\` en fin de ligne permet d'écrire une longue commande sur plusieurs lignes.

Conseil : les dossiers `logs`, `temp` et `work` doivent rester en écriture, Tomcat y écrit pendant l'exécution. Évite aussi `:ro` sur webapps : par défaut Tomcat y décompresse les WAR et fait du déploiement automatique. Pour le passer en lecture seule, il faut désactiver `unpackWARs` et l'auto-déploiement, ou déployer l'application déjà décompressée.

Volume anonyme

Si on ne précise que le chemin dans le conteneur (sans nom ni chemin hôte avant les deux-points), Docker crée quand même un volume — mais avec un identifiant généré automatiquement plutôt qu'un nom choisi. Il est donc plus difficile à retrouver et à réutiliser ensuite qu'un volume nommé.

```
docker run -v <chemin_conteneur> <image>
```

Crée un volume anonyme (nom généré par Docker) et le monte à ce chemin.

```
docker run -v /my-app/data mon-service:2.0.1
```

Exemple : Docker crée et gère seul le volume, sans nom choisi.

Bind mount

Autre façon de monter des données : au lieu d'un volume géré par Docker, on relie directement un dossier de la machine hôte au conteneur. C'est vous qui choisissez et gérez cet emplacement (pratique en développement pour accéder directement aux fichiers), contrairement au volume nommé dont l'emplacement est géré par Docker.

```
docker run -v <chemin_hôte>:<chemin_conteneur> <image>
```

Monte un dossier de l'hôte (chemin absolu ou relatif) directement dans le conteneur.

```
docker run -v ~/mon-app/data:/my-app/data mon-service:2.0.1
```

Exemple : monte le dossier ~/mon-app/data de l'hôte dans le conteneur.

```
docker run -p 8082:8082 --name mon-service -e SPRING_PROFILES_ACTIVE=test -v "$(pwd)/my-app-logs:/app/logs" mon-service:2.0.1
```

Exemple : les logs de l'application sont écrits directement dans le dossier `my-app-logs` du répertoire courant de l'hôte, consultables sans passer par Docker. `$(pwd)` est remplacé par le chemin absolu du dossier courant ; les guillemets protègent le chemin s'il contient des espaces. Si le dossier n'existe pas, `-v` le crée sur l'hôte.

Conseil : `$(pwd)` fonctionne sous Linux, macOS et Git Bash. En PowerShell, utilise `${PWD}` ; en invite de commandes Windows, `%cd%`. Sous Git Bash, si le chemin côté conteneur est mal converti (ex. `C:/Program Files/Git/app/logs`), préfixe la commande par `MSYS_NO_PATHCONV=1`.

```
ls my-app-logs          # 1. le fichier de log (ex. mon-service.log) est visible sur l'hôte
docker stop mon-service # 2. arrête le conteneur
docker rm mon-service   # 3. supprime le conteneur
ls my-app-logs          # 4. le fichier de log est toujours là
```

Vérification : les fichiers écrits par le conteneur dans un bind mount se trouvent directement dans le dossier de l'hôte et survivent à la suppression du conteneur. Un nouveau conteneur lancé avec le même `-v` les retrouvera.

Conseil : `docker rm -f mon-service` arrête et supprime le conteneur en une seule commande.

Bind mount pour un conteneur de base de données

Cas d'usage courant : relier le dossier de données d'une base (ici MySQL) à un dossier de l'hôte, pour que les bases survivent à la suppression du conteneur et restent visibles directement sur l'hôte.

```
docker run --name mon-mysql -e MYSQL_ROOT_PASSWORD=root_password -d -v "$(pwd)/mysql-data:/var/lib/mysql" mysql:9
```

Exemple : persister une base MySQL sur l'hôte. `/var/lib/mysql` est le dossier où MySQL stocke ses données ; en le reliant au dossier `mysql-data` du répertoire courant, les bases survivent à la suppression du conteneur et sont retrouvées par un nouveau conteneur lancé avec le même `-v`.

Conseil : au premier lancement, le dossier `mysql-data` doit être vide pour que MySQL initialise la base (le mot de passe `root` n'est pris en compte qu'à ce moment-là). Sous Windows et macOS, un volume nommé (`-v mysql-data:/var/lib/mysql`) est souvent plus rapide qu'un bind mount pour une base de données.

```
docker exec -it mon-mysql sh # 1. entre dans le conteneur
mysql -u root -p # 2. se connecte en root (demande le mot de passe)
show databases; # 3. liste les bases existantes
create database ma_base; # 4. crée une nouvelle base
exit # 5. quitte le client mysql
exit # 6. quitte le shell du conteneur
```

Suite de l'exemple : créer une base dans le conteneur MySQL. Elle est écrite dans `/var/lib/mysql`, donc dans le dossier `mysql-data` de l'hôte. Après `docker rm -f mon-mysql` puis le même `docker run ... -v "$(pwd)/mysql-data:/var/lib/mysql" ...`, `show databases`; affiche toujours `ma_base` : les données ont survécu au conteneur.

Conseil : `docker exec -it mon-mysql mysql -u root -p` ouvre directement le client MySQL, sans passer par le shell du conteneur.

```
ls mysql-data # sur l'hôte : un dossier par base + fichiers MySQL
```

Vérification côté hôte : le dossier `mysql-data` contient un sous-dossier par base (`ma_base`, `mysql`, `sys`, `performance_schema`) ainsi que les fichiers internes de MySQL (`ibdata1`, `binlog.*`, `undo_*`, certificats `*.pem...`). Relancer ensuite le même `docker run ... -v "$(pwd)/mysql-data:/var/lib/mysql" mysql:9` redémarre MySQL sur ces données existantes.

Conseil : ne modifie pas ces fichiers à la main, et ne partage pas le même dossier entre deux conteneurs MySQL lancés en même temps : la base serait corrompue.

tmpfs mount (Linux uniquement)

Monte un dossier directement en mémoire vive de l'hôte : rien n'est jamais écrit sur disque, les données disparaissent à l'arrêt du conteneur et ne sont partagées ni avec l'hôte ni avec d'autres conteneurs. Utile pour des fichiers temporaires ou sensibles (logs éphémères, caches).

```
docker run --tmpfs <chemin_conteneur> <image>
```

Monte le chemin donné en mémoire (tmpfs), Linux uniquement.

```
docker run -p 8082:8082 --name mon-service --tmpfs /app/logs mon-service:2.0.1
```

Exemple : l'application tourne normalement (port publié, conteneur nommé), mais ses logs dans `/app/logs` vivent en mémoire, jamais sur disque. Ils disparaissent dès que le conteneur s'arrête.

Conseil : « Linux uniquement » concerne les conteneurs Linux. Avec Docker Desktop sous Windows ou macOS, `--tmpfs` fonctionne aussi, car les conteneurs tournent dans une machine virtuelle Linux.

```
docker ps # 1. vérifie que le conteneur tourne
docker inspect mon-service | less # 2. cherche HostConfig.Tmpfs dans le JSON
docker exec -it mon-service sh # 3. entre dans le conteneur
mount | grep /app/logs # 4. affiche « tmpfs on /app/logs type tmpfs »
exit # 5. quitte le conteneur
```

Vérification : dans la sortie de `docker inspect` (parcourue avec `less`), la section `HostConfig` contient `"Tmpfs": { "/app/logs": "" }`, ce qui confirme que ce chemin est monté en mémoire. À l'intérieur du conteneur, `mount` liste tous les points de montage : la ligne `tmpfs on /app/logs type tmpfs (rw,nosuid,nodev,noexec,relatime)` le confirme aussi. Les options entre parenthèses sont celles appliquées par défaut par `--tmpfs` : lecture-écriture (`rw`), mais aucun programme ne peut être exécuté depuis ce dossier (`noexec`). Sans `grep`, la liste est longue (`systeme`, `/proc`, `/sys...`).

Conseil : `docker inspect -f '{{json .HostConfig.Tmpfs}}' mon-service` affiche uniquement ce champ, sans avoir à parcourir tout le JSON. Pour limiter la mémoire utilisée, précisez une taille : `--tmpfs /app/logs:size=64m`.

Astuce : inspecter le contenu d'un volume

On peut monter un volume existant dans un conteneur jetable, juste pour regarder ce qu'il contient, sans toucher au conteneur qui l'utilise habituellement.

```
docker run --rm -it -v my-app-volume:/data alpine sh
```

--rm supprime ce conteneur temporaire à la sortie, -it ouvre un shell interactif, et alpine est une image minimale pratique pour ce genre d'exploration ponctuelle. Une fois dedans, ls /data montre le contenu du volume.

Réseaux

Le réseau d'un conteneur dépend de son **driver** réseau, choisi avec --network au lancement (ou -d à la création d'un réseau). Docker en propose cinq principaux, selon que les conteneurs tournent sur une seule machine ou plusieurs, et selon le niveau d'isolation voulu.

```
docker network ls
```

Liste les réseaux existants. Par défaut, Docker en crée trois : bridge, host et none.

Bridge

Sur le réseau bridge par défaut : **chaque conteneur reçoit une adresse IP privée** ; les conteneurs peuvent se parler entre eux, mais **uniquement par adresse IP** (pas par nom) ; et pour être joignable depuis l'extérieur du réseau bridge (l'hôte, une autre application), un conteneur doit **publier un port** avec -p.

```
docker run --network bridge <image>
```

Réseau **par défaut** : plusieurs conteneurs tournent sur la même machine, dans un réseau privé isolé de l'hôte. Pour y accéder depuis l'hôte, on publie des ports avec -p. Sans --network, c'est ce driver qui est utilisé.

Conseil : sur un réseau bridge créé soi-même (docker network create <réseau>), les conteneurs se joignent par leur nom (DNS intégré), ce qui n'est pas le cas sur le bridge par défaut.

```
docker run -d --name produits -p 8081:8080 produits:1.0
```

```
docker run -d --name commandes -p 8082:8080 commandes:1.0
```

Exemple : deux applications sur le même hôte, toutes deux sur le réseau bridge par défaut. Chacune écoute sur le port 8080 **dans son conteneur**, mais elles sont publiées sur des ports hôte différents (8081 et 8082), car deux conteneurs ne peuvent pas publier le même port hôte.

- Entre conteneurs : commandes appelle produits via son IP privée et le **port conteneur**, ex.

http://<IP_products>:8080/products.

- Depuis l'extérieur : une autre application passe par l'hôte et le **port publié**, ex. http://<hôte>:8082 pour commandes.

- Plusieurs instances d'une même image peuvent tourner côte à côte : chacune a sa propre IP et son propre port hôte.

```
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' <container>
```

Affiche l'adresse IP privée d'un conteneur sur son réseau (ex. 172.17.0.2), celle à utiliser pour l'appeler depuis un autre conteneur du bridge par défaut.

Conseil : cette IP peut changer à chaque redémarrage du conteneur. Pour des appels stables entre conteneurs, préfère un réseau bridge personnalisé où l'on appelle les conteneurs par leur nom.

Host

Réservé aux **conteneurs Linux**. Le conteneur utilise directement le réseau de la machine hôte : il n'a ni réseau séparé ni adresse IP propre (il utilise celle de l'hôte), et il n'y a **pas besoin de publier de port** : un port ouvert par l'application est directement un port de l'hôte.

```
docker run --network host <image>
```

Le conteneur partage directement le réseau de la machine hôte : pas d'IP propre, ses ports sont ceux de l'hôte, donc `-p` est inutile. Voir aussi l'exemple MySQL dans « Options avancées de run ».

```
docker run -d --name produits --network host produits:1.0 # 8080
docker run -d --name commandes --network host -e server.port=8081 commandes:1.0 # 8081
```

Exemple : deux applications en réseau host, sans aucun `-p`. Comme elles partagent les ports de l'hôte, elles doivent écouter sur des **ports différents** (ici 8081 via `server.port` pour l'application Spring Boot).

- Depuis une autre application sur l'hôte : `http://localhost:8080/products` et `http://localhost:8081/orders`.
- Depuis une autre machine : `http://<hôte>:8081`, directement sur le port de l'application.

Conseil : en mode host, `-p` est ignoré, et si deux conteneurs (ou un conteneur et un programme de l'hôte) veulent le même port, le second ne pourra pas démarrer son serveur. Avec Docker Desktop (Windows, macOS), ce mode ne se comporte pas comme sous Linux ; les versions récentes le proposent en option à activer dans les paramètres.

None

```
docker run --network none <image>
```

Aucun réseau : le conteneur n'a que l'interface locale (`localhost`) et ne peut joindre ni l'hôte, ni Internet, ni d'autres conteneurs. Utile pour un traitement isolé qui n'a besoin d'aucun accès réseau.

Overlay

```
docker network create -d overlay <réseau>
```

Relie des conteneurs qui tournent sur des machines différentes, comme s'ils étaient sur le même réseau.

Conseil : ce driver s'utilise avec Docker Swarm ; il faut d'abord initialiser le cluster avec `docker swarm init`.

Macvlan

```
docker network create -d macvlan --subnet <sous-réseau> --gateway <passerelle> -o parent=
<interface> <réseau>
```

Le conteneur reçoit sa propre adresse MAC et sa propre IP : il apparaît comme une machine distincte sur le réseau physique. Utile pour des applications qui doivent être jointes directement sur le réseau local.

Conseil : fonctionne sur un hôte Linux. Par défaut, l'hôte lui-même ne peut pas joindre ses conteneurs macvlan.